

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

Một số phép xử lý ma trận của NumPy và Scipy:

```
✓from scipy import linalg # Tải gói linalg của scipy vào bộ nhớ (để sử dụng)
import numpy as np # Tải gói numpy vào sử dụng với tên gọi là np
a = np.array([1, 2, 3])
b = np.array([(1+9j, 2j, 3j), (4j, 5j, 6j)]) # số phức
c = np.array([(0.5, 1.5, 10), (3,2,1) ], [(6,5,4), (7,8,9)]) # ma trận số thực
# Các kiểu khai báo và khởi tạo ma trận:
A = np.matrix(np.random.random( (2,2) ) )
B = np.asmatrix(b) # Chuyển b thành dạng ma trận
C = np.matrix(np.random.random( (10,5) ) )
D = np.matrix([ [4, 3], [2, 6] ])
F = np.eye(3, k=1) # tạo ma trận đường chéo. 3 là ma trận 3x3, k=1 là đường
# chéo nằm phía trên đường chéo chính (k = 0)
print("Ma trận A:\n", A)
print("Ma trận B:\n", B)
print("Ma trận C:\n", C)
print("Ma trận D:\n", D)
print("Ma trận F:\n", F)
```

Kết quả:

```
Ma trận A:
[[0.43141515 0.10383282]
 [0.96716525 0.79369362]]
Ma trận B:
[[1.+9.j 0.+2.j 0.+3.j]
 [0.+4.j 0.+5.j 0.+6.j]]
Ma trận C:
[[0.20455703 0.28924734 0.9448908 0.85996777 0.24373272]
 [0.30625695 0.72425141 0.58029338 0.02112096 0.53606008]
 [0.39746883 0.8419533 0.54149223 0.95758362 0.93517946]
 [0.33136989 0.6357499 0.63022968 0.98634738 0.98127723]
 [0.43932291 0.12411127 0.06011569 0.22844234 0.6009662 ]
 [0.32761392 0.52445224 0.87551793 0.48033301 0.44403405]
 [0.43362093 0.42147477 0.40377978 0.05388437 0.5802251 ]
 [0.9853895 0.02313016 0.42032195 0.15919271 0.65336631]
 [0.89361479 0.66232523 0.38532428 0.08830734 0.61422009]
 [0.7557203 0.03021515 0.4044672 0.89434631 0.14679343]]
Ma trận D:
[[4 3]
 [2 6]]
Ma trận F:
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 0.]
```

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# Các phép xử lý đơn giản
# Xem hạng ma trận:
print("Hạng của ma trận C:", np.linalg.matrix_rank(C))
# Tính ma trận nghịch đảo:
print("Ma trận nghịch đảo của A:\n", A.I)
print("Ma trận nghịch đảo của A (sử dụng scipy.linalg.inv):\n", linalg.inv(A))
# Định thức ma trận:
print("Định thức của A:", linalg.det(A))
# Chuyển đổi ma trận:
print("Chuyển vị của A:\n", A.T)
print("Chuyển vị liên hợp của A:\n", A.H)
```

Kết quả:

```
Hạng của ma trận C: 5
Ma trận nghịch đảo của A:
[[ 3.27988891 -0.42908259]
 [-3.9967495  1.78279594]]
Ma trận nghịch đảo của A (sử dụng scipy.linalg.inv):
[[ 3.27988891 -0.42908259]
 [-3.9967495  1.78279594]]
Định thức của A: 0.24198795896569883
Chuyển vị của A:
[[0.43141515 0.96716525]
 [0.10383282 0.79369362]]
Chuyển vị liên hợp của A:
[[0.43141515 0.96716525]
 [0.10383282 0.79369362]]
```

```
# Giải các loại phương trình tuyến tính:
print("Nghịệm của phương trình Ax = b là:\n", linalg.solve(A, b))

E = np.matrix(a).T
print(linalg.lstsq(F, E)) # Giải phương trình xấp xỉ F*x = E
```

Kết quả:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

Nghiệm của phương trình $Ax = b$ là:

```
[ [ 3.27988891+27.80266983j  0.          +4.41436488j
    0.          +7.2651712j ]
 [-3.9967495 -28.83956172j -0.          +0.92048071j
    0.          -1.29347284j]]
(array([[0.],
        [1.],
        [2.]]), array([], dtype=float64), 2, array([1., 1., 0.]))
```

Các hàm trên ma trận

```
print("Tổng của A và D:\n", np.add(A, D))

print("Hiệu của A và D:\n", np.subtract(A, D))

print("Thương của A và D:\n", np.divide(A, D))

print("Tích của A và D:\n", A @ D)

print("Tích của D và A:\n", np.multiply(D, A))

print("Tích của A và D:\n", np.dot(A, D))

print("Tích của A và D:\n", np.vdot(A, D))
```

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
✓from scipy import linalg # Tải gói linalg của scipy vào bộ nhớ (để sử dụng)
import numpy as np # Tải gói numpy vào sử dụng với tên gọi là np
a = np.array([1, 2, 3])
b = np.array([(1+9j, 2j, 3j), (4j, 5j, 6j)]) # số phức
c = np.array([(0.5, 1.5, 10), (3,2,1) ], [(6,5,4), (7,8,9)]) # ma trận số thực
# Các kiểu khai báo và khởi tạo ma trận:
A = np.matrix(np.random.random( (2,2) ) )
B = np.asmatrix(b) # Chuyển b thành dạng ma trận
C = np.matrix(np.random.random( (10,5) ) )
D = np.matrix([ [4, 3], [2, 6] ])
F = np.eye(3, k=1) # tạo ma trận đường chéo. 3 là ma trận 3x3, k=1 là đường
# chéo nằm phía trên đường chéo chính (k = 0)
print("Ma trận A:\n", A)
print("Ma trận B:\n", B)
print("Ma trận C:\n", C)
print("Ma trận D:\n", D)
print("Ma trận F:\n", F)
```

Kết quả:

```
Tổng của A và D:
[[4.43141515  3.10383282]
 [2.96716525  6.79369362]]
Hiệu của A và D:
[[-3.56858485 -2.89616718]
 [-1.03283475 -5.20630638]]
Thương của A và D:
[[0.10785379  0.03461094]
 [0.48358263  0.13228227]]
Tích của A và D:
[[1.93332624  1.91724237]
 [5.45604826  7.6636575  ]]
Tích của D và A:
[[1.7256606   0.31149846]
 [1.93433051  4.76216174]]
Tích của A và D:
[[1.93332624  1.91724237]
 [5.45604826  7.6636575  ]]
Tích của A và D:
[[1.7256606]]
```

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# Hàm lũy thừa và logarith ma trận:  
print("Ma trận lũy thừa của A:\n", linalg.expm(A))  
  
print("Ma trận logarith của A:\n", linalg.logm(A))
```

Kết quả:

```
Ma trận lũy thừa của A:  
[[1.62772948  0.1958687 ]  
 [1.82444623  2.31112623]]  
Ma trận logarith của A:  
[[-1.0502338   0.19535381]  
 [ 1.81965026 -0.36863351]]
```

Ma Trận Hội Tụ:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# Ma trận phân kỳ:
import numpy as np
# Tính ma trận A:
A = np.array([ [0,1], [1,0] ])
print("Ma trận A:\n", A)
temp = A.dot(A) # tính toán lần thứ 1
print("Ma trận A^2:\n", temp)
k= 6
for i in range(k-1):
    temp = temp.dot(A)
    print (temp)
    print('---')
# Tính ma trận B:
B = np.array([ [0,-1], [-1,0] ])
print("Ma trận B:\n", B)
temp = B.dot(B) # tính toán lần thứ 1
print("Ma trận B^2:\n", temp)
k= 5
for i in range(k-1):
    temp = temp.dot(B)
    print (temp)
    print('---')
```

Kết quả:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

Ma trận A:

$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

Ma trận A^2 :

$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

Ma trận B:

$\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$

Ma trận B^2 :

...

$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# Ma trận hội tụ - Convergent matrix:
C = np.array([ [1, 0, 0], [0, 0.5, 1], [0, 0, 0.5] ])
print("Ma trận C:\n", C)
temp = C.dot(C)
print("Ma trận C^2:\n", temp)
k= 1000
for i in range(k-1):
    temp = temp.dot(C)
print (temp)
# Sau đó, thực hiện thêm 1 lần tích 1000 lần nữa:
k= 1000
for i in range(k-1):
    temp = temp.dot(C)
print(temp)
```

Kết quả:

```
Ma trận C:
[[1.  0.  0. ]
 [0.  0.5 1. ]
 [0.  0.  0.5]]
Ma trận C^2:
[[1.  0.  0. ]
 [0.  0.25 1. ]
 [0.  0.  0.25]]
[[1.00000000e+000 0.00000000e+000 0.00000000e+000]
 [0.00000000e+000 4.66631809e-302 9.34196882e-299]
 [0.00000000e+000 0.00000000e+000 4.66631809e-302]]
[[1. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Ma Trận Markov:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# Ma trận Markov - Markov matrix:
import numpy as np
M = np.array([ [0.8, 0.3], [0.2, 0.7]])
MM = M.dot(M) # tính M2, nghĩa là M^2
print("Ma trận MM^2:\n", MM)

MM = M.dot(M) # tính M3
print("Ma trận MM^3:\n", MM)
for i in range(100): # tính các M^ khác.
    MM = MM.dot(M)
print (MM) # kết luận xu hướng của M^n khi n tiến tới vô cùng
```

Kết quả:

```
Ma trận MM^2:
[[0.7  0.45]
 [0.3  0.55]]
Ma trận MM^3:
[[0.7  0.45]
 [0.3  0.55]]
[[0.6  0.6]
 [0.4  0.4]]
```

```
M = np.array([ [0.5, 0.5], [0.5, 0.5]])
MM = M.dot(M) # tính M2, nghĩa là M^2
print("Ma trận MM^2:\n", MM)

MM = M.dot(M) # tính M3
print("Ma trận MM^3:\n", MM)
for i in range(100): # tính các M^ khác.
    MM = MM.dot(M)
print (MM) # kết luận xu hướng của M^n khi n tiến tới vô cùng
```

Kết quả:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
Ma trận MM^2:  
[[0.5 0.5]  
 [0.5 0.5]]  
Ma trận MM^3:  
[[0.5 0.5]  
 [0.5 0.5]]  
[[0.5 0.5]  
 [0.5 0.5]]
```

Phân rã ma trận LU:

```
import numpy as np  
from scipy import linalg  
M = np.array([ [0.8, 0.3], [0.2, 0.7]])  
P, L, U = linalg.lu(M) # lệnh linalg.lu sẽ tách ma trận M thành 3 ma trận P, L và U  
print("Ma trận P:\n", P)  
print("Ma trận L:\n", L)  
print("Ma trận U:\n", U)  
# Thử lại với phép nhân LU có bằng ma trận M ban đầu hay không:  
print("Tích ma trận L và U:\n", L.dot(U))
```

Kết quả:

```
Ma trận P:  
[[1. 0.]  
 [0. 1.]]  
Ma trận L:  
[[1. 0. ]  
 [0.25 1. ]]  
Ma trận U:  
[[0.8 0.3 ]  
 [0. 0.625]]  
Tích ma trận L và U:  
[[0.8 0.3]  
 [0.2 0.7]]
```

Bài toán ứng dụng 2 – Bài toán loan tin:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
import numpy as np
A = np.array([ [0,1,0,1],[0,0,1,0],[1,0,0,1],[1,1,0,0]])
temp = A.dot(A) # lúc này temp = A^2
print("Ma trận A^2:\n", temp)

temp = temp.dot(A) # lúc này temp = A^3
print("Ma trận A^3:\n", temp)

A = np.array([ [0,1,0,1],[0,0,1,0],[1,0,0,1],[1,1,0,0]])
sumA = A
temp = A.dot(A)
k = 3
sumA = sumA + temp
for i in range(1, k-1):
    temp = temp.dot(A)
    sumA = sumA + temp
print("temp:\n", temp)
print("Ma trận S = A + A^2 + A^3:\n", sumA)
```

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

Kết quả:

```
Ma trận A^2:  
[[1 1 1 0]  
 [1 0 0 1]  
 [1 2 0 1]  
 [0 1 1 1]]  
Ma trận A^3:  
[[1 1 1 2]  
 [1 2 0 1]  
 [1 2 2 1]  
 [2 1 1 1]]  
temp:  
[[1 1 1 2]  
 [1 2 0 1]  
 [1 2 2 1]  
 [2 1 1 1]]  
Ma trận S = A + A^2 + A^3:  
[[2 3 2 3]  
 [2 2 1 2]  
 [3 4 2 3]  
 [3 3 2 2]]
```

Bài tập chương 3:

Câu 1: Hãy viết chương trình bằng Python hoặc sử dụng thư viện numpy/sympy để:

- Tự chọn một ma trận khả nghịch 3x3. Sinh viên chứng minh ma trận đó khả nghịch (tồn tại ma trận nghịch đảo)
- Nhập họ và tên hoặc mã số sinh viên (của sinh viên).
- Mã hóa họ và tên hoặc mã số sinh viên (của sinh viên).
- Thực hiện giải mã với ma trận được chọn.

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
# a.  
import numpy as np  
A = np.array([[2, 1, 1],[1, 2, 1],[1, 1, 2]])  
  
# Kiểm tra khả nghịch  
det_A = np.linalg.det(A)  
print("Định thức của A =", det_A)  
  
A_inv = np.linalg.inv(A)  
print("Ma trận nghịch đảo của A:\n", A_inv)
```

Kết quả:

```
Định thức của A = 4.0  
Ma trận nghịch đảo của A:  
[[ 0.75 -0.25 -0.25]  
 [-0.25  0.75 -0.25]  
 [-0.25 -0.25  0.75]]
```

```
# b.  
name = "Tran Nhat Thinh"
```

```
# c.  
# Chuyển từng ký tự trong tên sang mã ASCII  
ascii_codes = [ord(c) for c in name]  
# Bổ sung 0 để số phần tử chia hết cho 3  
while len(ascii_codes) % 3 != 0:  
    |   ascii_codes.append(0)  
# Chuyển danh sách ASCII thành ma trận (mỗi hàng 3 phần tử)  
matrix_text = np.array(ascii_codes).reshape(-1, 3)  
# Mã hóa bằng ma trận A  
encoded = matrix_text @ A  
print("Ma trận mã hóa:\n", encoded)
```

Kết quả:

CHƯƠNG 3: MA TRẬN VÀ HỆ PHƯƠNG TRÌNH TUYẾN TÍNH

```
Ma trận mã hóa:  
[[379 409 392]  
 [330 252 298]  
 [421 414 433]  
 [252 304 324]  
 [424 429 423]]
```

```
# d.  
# Ma trận nghịch đảo  
A_inv = np.linalg.inv(A)  
  
# Giải mã  
decoded = encoded @ A_inv  
decoded = np.round(decoded).astype(int)  
  
# Chuyển ASCII về chuỗi  
decoded_name = ""  
for row in decoded:  
    for num in row:  
        if num != 0:  
            decoded_name += chr(num)  
  
print("Chuỗi sau khi giải mã:", decoded_name)
```

Kết quả:

```
Chuỗi sau khi giải mã: Tran Nhat Thinh
```